Inductive link prediction for missing medical codes in personal health knowledge graphs

Ömer Durukan Kılıç[1,*], Ensar Emir Erol[1] and Remzi Çelebi[1]

[1]*Maastricht University, Department of Advanced Computing Sciences, Institute of Data Science, Paul-Henri Spaaklaan 1, 6221 GS, Maastricht, Limburg, The Netherlands*

Abstract

Electronic health records (EHR) frequently suffer from incompleteness which limits their clinical utility. To address this, we present a data imputation implementation that employs an inductive Graph Neural Network (GNN) model to predict missing values within integrated personal EHRs in the form of knowledge graphs (KGs). To overcome the limitations of traditional transductive learning, we use the NodePiece, an inductive link prediction model that can generalize and infer links for patients never seen during training. The experiments on a subset of the MIMIC-III clinical dataset proved that the model is highly effective for clinical measurements, identifying the correct code within the top-5 suggestions (Hit@5) in over 99% of cases. However, while the model successfully reduced the search space for diagnostic codes, it struggled with ranking confidence and performed poorly on sparse procedure codes, indicating a need for further refinement in specific domains. This service, which was integrated into the AIDAVA prototype, offers a scalable solution to impute and enrich patient health data.

Keywords

Data Imputation, Knowledge Graphs, Inductive Learning, Link Prediction, Graph Neural Networks, Electronic Health Records

1. Introduction

Electronic health records (EHRs) have great potential to advance personalized medicine and clinical research. However, EHR data are often noisy, heterogeneous, and stored in unstructured formats, limiting their (re)use. Among many issues in digital health records, a common challenge is missing patient information [1]. Critical data such as diagnosis or procedure codes, may be absent due to various factors ranging from system failures and insufficient documentation practices to the privacy concerns that restrict information sharing across systems [2]. These issues hinder data analysis and significantly reduce the overall quality and usability of healthcare data.

The AIDAVA[1] (AI-powered Data Curation & Publishing Virtual Assistant) project aims to solve data integration and interoperability challenges by developing a semi-automated orchestration process for cleaning, structuring, and standardizing health records. To generate higher-quality and complete patient data, we employ a data imputation strategy that predicts missing essential information, particularly for medical codes. Current statistical approaches often lack the capacity to understand the complex, non-linear structure of patient histories [3]. This creates a need for alternatives that leverage machine learning (ML) and semantic web technologies. By modeling patient records as knowledge graphs (KGs), our model can capture the intricate, non-linear dependencies between medical events that flat tabular formats obscure. In this graph-based approach, missing data points are re-conceptualized as "missing links" between a patient and a medical concept. This data imputation model, integrated into the AIDAVA prototype, leverages inductive link prediction to infer missing links. This choice reflects the realities of

17th *International SWAT4HCLS Conference, March 23–26, 2026, Amsterdam, The Netherlands*

*Corresponding author.

omer.kilic@maastrichtuniversity.nl (Ö. D. Kılıç); ensar.erol@maastrichtuniversity.nl (E. E. Erol); remzi.celebi@maastrichtuniversity.nl (R. Çelebi)

https://github.com/omerdurukankilic (Ö. D. Kılıç); https://github.com/ensaremirerol (E. E. Erol); https://github.com/rcelebi (R. Çelebi)

0009-0000-7823-4753 (Ö. D. Kılıç); 0000-0002-5739-8860 (E. E. Erol); 0000-0001-7769-4272 (R. Çelebi)

© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

[1]https://www.aidava.eu/

current healthcare settings, where ML models cannot be continuously retrained and re-deployed as new patient data enter the system. Consequently, the model must generalize to previously unseen patient data, necessitating a shift away from traditional learning paradigms and a clear distinction between transductive and inductive approaches.

Transductive learning operates under a ”closed-world” assumption, where predictions are made only for entities that were present and observed during the training phase [4]. While effective for static datasets, it fails in dynamic healthcare environments, as imputing data for new patients requires costly and impractical retraining [5]. In contrast, inductive learning builds general rules based on patterns in the data, rather than simply memorizing specific patients [6]. This capability allows the system to process new patients immediately, avoiding the common failure where a model cannot handle someone it has not seen before. By aligning a new patient's connections with existing medical codes, the model can instantly understand them and predict missing information for individuals completely absent from the original training data [6]. Switching from memorization to reasoning is essential for building reliable, real-time services that keep working as patient records grow.

To address the gap in translating inductive learning architectures into clinical practice, this study tackles a primary research problem: Existing imputation methods rely on static patient representations that cannot generalize to new individuals without computationally expensive re-training. This limitation creates a critical barrier, rendering current approaches ineffective for real-time support in dynamic healthcare environments where new patients are continuously admitted.

In this work, we mapped EHR data to a knowledge graph representation and applied inductive link prediction for imputing medical codes. To validate this approach, we conducted extensive experiments using a subset of the MIMIC-III clinical database [7], stratifying our evaluation across distinct clinical domains of laboratory tests, diagnoses, and medical operations. Our initial results indicated that while the model was often able to identify the correct medical code among its top suggestions, it frequently struggled to assign a high confidence score to a single definitive answer. We observed that the system often viewed several codes as equally likely correct answers, rather than confidently distinguishing the best one from similar alternatives. To address this uncertainty, we conducted further experiments where we enriched the input knowledge graph with a taxonomy of diagnostic codes, ICD-9 to be precise. This enrichment proved helpful for predicting the other entity types, e.g. laboratory tests and medical procedures, allowing the model to be much more certain in its rankings. However, for diagnoses, adding this vast amount of information about their hierarchy had a complex trade-off: It introduced too many new possible codes to the model. Without a specific patient history for these new concepts, the model struggled to narrow down the list of diagnoses. The findings highlight that while inductive prediction is viable, specific mechanisms are still required to fully overcome challenges unique to clinical records.

2. Background

EHRs have grown from simple digital files into complex systems that track a patient's entire medical history and hospital stay. While these records contain a vast amount of invaluable information, the quality of the data is often questionable. This limits the use of this data for important downstream tasks, such as clinical decision-support or identifying patient risks [8]. It is important to understand the reasons of missing data in the first place and why standard methods often fail to address the problem.

2.1. Incompleteness in Electronic Health Records

Missing information is not just an accident in healthcare; it is a natural part of how medical data is collected. Sometimes sensors fail, there is a lack of systemic foundation for proper record-keeping, or data is missing because of human decisions [2]. For example, a doctor will usually only order a specific test *only if* a patient shows symptoms that warrant it [9], leading to lack of measurements or procedures for a patient. In the literature, missing data is grouped into three categories:

1. **Missing Completely at Random:** The missing data has no pattern and is unrelated to the patient's health. This is rare in healthcare.
2. **Missing at Random:** The missing data can be explained by other available information. For instance, a missing pregnancy test result is expected if the patient is male.
3. **Missing Not at Random:** This is the most difficult type. Here, the fact that data is missing provides a clue about the value itself. For example, if a patient's temperature is not recorded, it likely means they did not have a fever, therefore, the measurement was never made.

Because "Missing Not at Random" is so common in medicine, simply ignoring incomplete records leads to the loss of valuable information and biased the results [10]. In literature, methods have been developed to derive implicit information from data when it is purposefully missing, or more importantly, "imputing" data where there is none due to poor record-keeping practices.

2.2. Data Imputation Methods

Researchers used statistical methods to predict missing values in previous works. While these methods work for simple structures like spreadsheets, they struggle with the complexity of modern medical records. Table 1 outlines these common methods and why they would under-perform in a healthcare context.

Table 1
Comparison of traditional data imputation methods and their limitations specific to the healthcare domain.

Method	Approach	Limitations for Healthcare Domain
Mean/Median	Fills gaps with the average value of the group.	Ignores individual patient differences. An "average" blood sugar level may be dangerous for a diabetic patient [11].
LOCF (Last Observation Carried Forward)	Uses the patient's last known value.	Assumes health is static. This fails in emergencies where a patient's condition changes rapidly [11].
MICE (Chain Equations)	Uses other variables to predict the missing one.	Too slow for datasets with thousands of medical codes; assumes simple linear relationships [12].
Matrix Factorization	Finds hidden patterns in the data matrix.	Cannot handle new patients easily; requires complex math to be re-done whenever new data arrives [13].

Newer methods use machine learning (ML) to improve upon simple statistics. These methods attempt to predict the missing value patterns by looking at correlations between different features in the data. A couple of prominent ML approaches are as follows:

K-Nearest Neighbors (kNN): This method finds the "k" most similar patients in the database and averages their values to fill in the missing gap. While intuitive, it is computationally very slow ($O(N^2)$). In a sparse medical database with thousands of codes, it is often impossible to find truly "similar" patients because everyone has a unique combination of rare diseases [14].

Random Forests: This approach trains a decision tree model for every missing variable, using all other variables as predictors. It is powerful because it can handle non-linear relationships (e.g., "Age" affects "Risk" differently if "Smoker" is true). However, like MICE, it becomes incredibly slow when applied to datasets with tens of thousands of diagnosis codes [11].
While these methods are an improvement, they still treat the data as a flat table. They often struggle to scale to the massive dimensionality of EHRs and fail to capture the sequential/inter-dependent structure of clinical patient data. Therefore, more recently, researchers have turned to deep learning to handle the scale and complexity problem. These models are designed to process massive datasets and find hidden, non-linear patterns that standard ML misses. Some examples are as follows:

Recurrent Neural Networks (RNNs): RNNs read data as a sequence, like a story. They are effective for patient history because they understand that the order of events matters (e.g., a diagnosis usually comes before a treatment) [3].

Autoencoders (Denoising & Variational): Such models work by compressing a patient's record into a small summary and then trying to reconstruct the original record from that summary. If trained correctly, they can fill in missing blanks during reconstruction [12].

Generative Adversarial Networks (GANs): These models, like GAIN [15], use two competing networks: a "Generator" that tries to create realistic fake data to fill gaps, and a "Discriminator" that tries to catch the fakes. This often produces very realistic results but is known to be unstable and difficult to train [16].
Despite their power, deep learning methods face significant drawbacks in clinical settings. First, they are often "black boxes," meaning it is hard to explain to a doctor why the model guessed a specific value [17]. Second, they often require massive datasets to work well. Finally, they typically treat medical codes as arbitrary numbers. They do not inherently understand that "Type 1 Diabetes" is medically related to "Insulin" unless they see that specific pattern many times in the data. They lack the medical common sense or semantic knowledge that a domain experts rely on.

2.3. Knowledge Graph Based Approaches

Instead of storing data in flat tables or treating codes as random numbers, a KG represents data as a semantic network. It can connect patients to their medical events, and connect those events to general medical knowledge (like the fact that "Viral Pneumonia" is a type of "Lung Disease"). This approach has two main benefits. First, it preserves context: a diagnosis is not just a standalone number but a node connected to related symptoms and treatments. Second, it simplifies the problem. In a graph, a missing data point is simply a "missing link" between two nodes. Therefore, filling in missing data becomes a task of *Link Prediction*, i.e. predicting which connections should exist based on the surrounding structure.

While knowledge graphs are powerful, the way they are trained matters depending on the problem. There are two main approaches: *transductive* and *inductive*. Transductive models work by memorizing the entities they see during training. They work on the "Closed World Assumption," i.e. other than the provided information nothing new will ever appear. Inductive models, on the other hand, learn general rules and patterns. This allows them to handle new, unseen items by looking at their features and context [5].

To highlight the difference, our previous attempt at link prediction can be mentioned. Initially, using the same data (i.e. MIMIC-III [7]) in this study, we trained a transductive model to predict missing medical codes. We split the data to train and test sets, then created numeric representations of them using knowledge graph embeddings, e.g. TransE [18], RotatE [19], DistMult [20], ComplEx [21], and QuatE [22]. Afterwards, a training loop memorized every single representation. During evaluation, we have observed several critical failures. First of all, the afore-mentioned problem with needing to re-train the model when a test patient with completely new entities arrived was apparent. This resulted in the model not being able to make accurate predictions as it was confined only to the information set it was trained on. Moreover, the transductive training tended to focus only on very common diseases due to them appearing more times in the data. For rare medical codes the model often failed to find the correct data to impute, defaulting to common ones that it had previously seen many times. These limitations empirically showed that the transductive learning is unsuitable for the real-life medical use (and the AIDAVA project), where patients continuously add new data and new medical concepts emerge over time. It would be costly and inefficient to retrain the entire system every time a patient visits a doctor. This necessitated the switch to an inductive model such as *NodePiece*.

NodePiece breaks a graph down into smaller, reusable parts called "tokens" to address these limitations. It then dynamically builds a representation for a patient based on these small representations. This

means that when a new patient enters the system, the model can immediately construct their data with the tokens at hand and make predictions without needing to be retrained [6]. It scales easily and can even handle partial data, making it ideal for a real-time, patient-centric system.

3. Methodology

We reformulate data imputation as a link prediction task over a clinical knowledge graph. Unlike traditional models that rely on memorizing specific patient identities, we employ an inductive GNN architecture designed to learn generalizable medical patterns. This approach allows the system to infer missing diagnostic or procedure codes for new patients based solely on their available clinical context, without the need for retraining. The pipeline consists of (i) KG construction and patient-level splitting, (ii) inductive model training with negative sampling, and (iii) evaluation on unseen patients. Table 2 details the model's outputs for different entity types.

Table 2
Raw clinical data categories (*Missing Variable*) are mapped to specific entities that best corresponds to them within the AIDAVA Reference Ontology (*Mapped Entity Type*). In MIMIC-III, laboratory tests, diagnoses, and medical operations are coded using *Logical Observation Identifiers Names and Codes* (LOINC), *The International Classification of Diseases, Ninth Revision* (ICD-9), and *Current Procedural Terminology* (CPT) terminologies, respectively. Our model is trained to output medical codes in their missing variable's corresponding terminology (*Output Medical Code*).

Missing Variable	Mapped Entity Type	Output Medical Code
Laboratory Test	`Measurement`	LOINC
Diagnosis	`ProblemCondition`	ICD-9
Medical Operation	`Procedure`	CPT

3.1. Data & Preprocessing

We use a KG derived from a subset of the MIMIC-III [7], containing 1001 patients with the high number of medical codes. The main reason for using a subset instead of the whole data is computational requirement for model training. A secondary one is to observe how skewed data affects such data imputation model: This subset has less code diversity as well as less instances for the model to train on, especially for medical procedures

The MIMIC-III Knowledge Graph, represented in RDF format, captures patient information including laboratory tests (i.e. `Measurement` entities), diagnoses/observations (i.e. `ProblemCondition` entities), and medical operations (i.e. `Procedure` entities). All medical codes are mapped to standardized medical vocabularies such as LOINC[2] for clinical measurements, ICD-9[3] for diagnoses, and CPT[4] for procedures. Figure 1 is a visualization of the MIMIC-III knowledge graph structure.

On a supporting note, adopting a knowledge graph structure for processing EHRs supports FAIR principles, specifically making data interoperable and reusable [23]. Unlike rigid tables, our use of standard medical definitions, presented in AIDAVA Reference Ontology, ensures the system is not locked to the MIMIC-III dataset. This flexibility improves transferability, allowing the model to work in other clinical settings that use standard codes. Beyond simple data storage, this approach allows the model to understand the meaning and relationships behind medical concepts, rather than treating them as arbitrary numbers.

[2]https://loinc.org/
[3]https://www.who.int/standards/classifications/classification-of-diseases
[4]https://www.ama-assn.org/practice-management/cpt/cpt-overview-and-code-approval
[5]https://github.com/AIDAVA-DEV/AIDAVA-Reference-Ontology
[6]https://www.biomedit.ch/rdf/sphn-ontology/sphn/2023/2

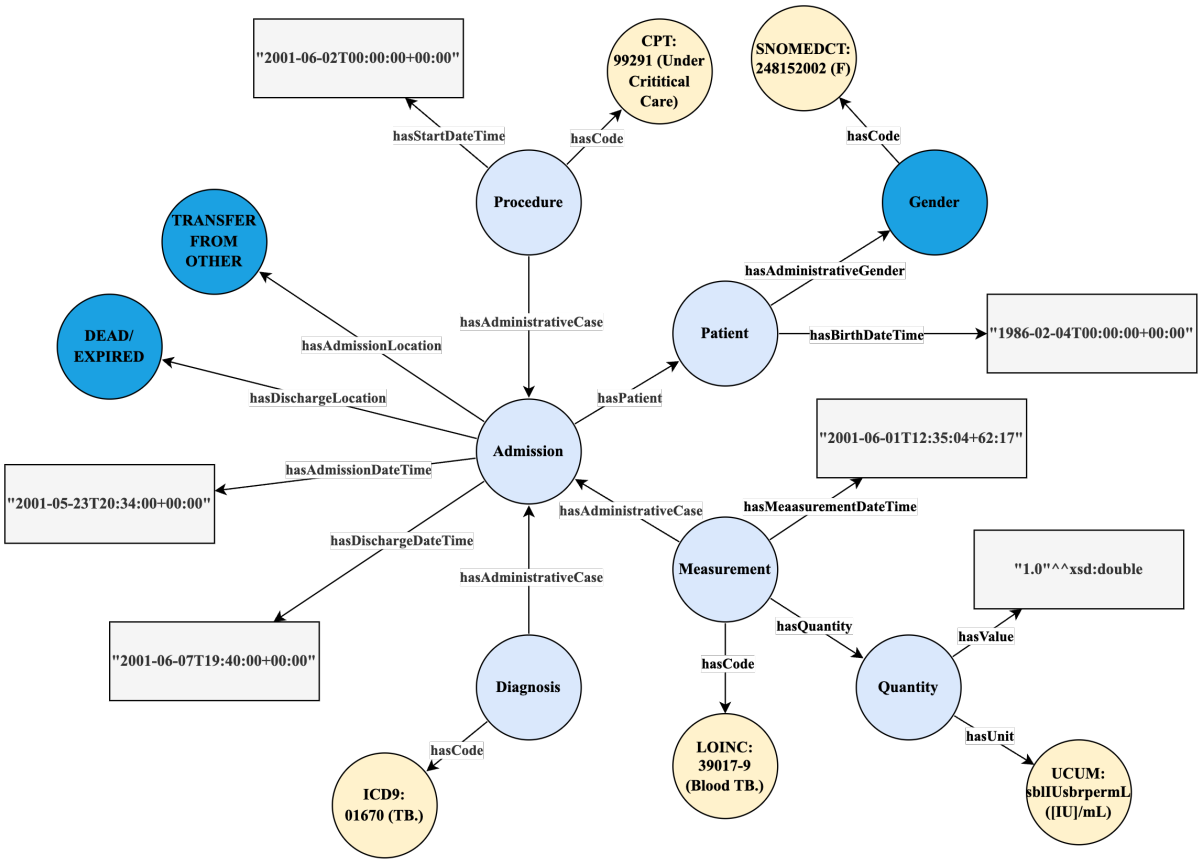

Figure 1: A visual representation of the MIMIC-III [7] Knowledge Graph utilized for training, testing, and the validation of the data imputation model. Concepts from the AIDAVA Reference Ontology[5], which extends an older version of SPHN RDF Schema[6], were used to represent entities and relations between them. Mapping of the MIMIC-III tables to AIDAVA Ontology was done at Maastricht University by AIDAVA researchers.

To evaluate inductive generalization, we apply patient-level splitting (80% train, 20% test), ensuring that all data for a patient appears in only one split. Importantly, all medical records associated with a given patient remain together in either the training or testing set, preventing information leakage that could artificially inflate performance metrics. In other words, the model is intended to learn a patient's characteristics and how it relates to missing codes since a single patient's data is not divided between the training and inference sets. Instead the patients are split, then their data is added to their respective sets; ensuring a patient context is never broken during training and validation. The splitting process generates four distinct datasets (see Figure 2):

1. **Training Graph:** Contains all medical information for patients designated for training, including their associated lab tests, observations, and procedures.
2. **Inductive Inference Graph:** From test patients, medical code links are removed to create an inductive inference graph. The removed links are divided equally into validation and test sets, preventing information leakage while preserving full patient context.
   a **Validation Graph:** Includes 50% of the removed medical codes, used during training to monitor the model's performance and trigger early stopping when improvement plateaus.
   b **Test Graph:** Contains the remaining 50% of removed medical codes, reserved for final performance evaluation. This graph is further divided by entity type (`Measurement`, `ProblemCondition`, and `Procedure`) to enable granular performance analysis across different clinical data categories.

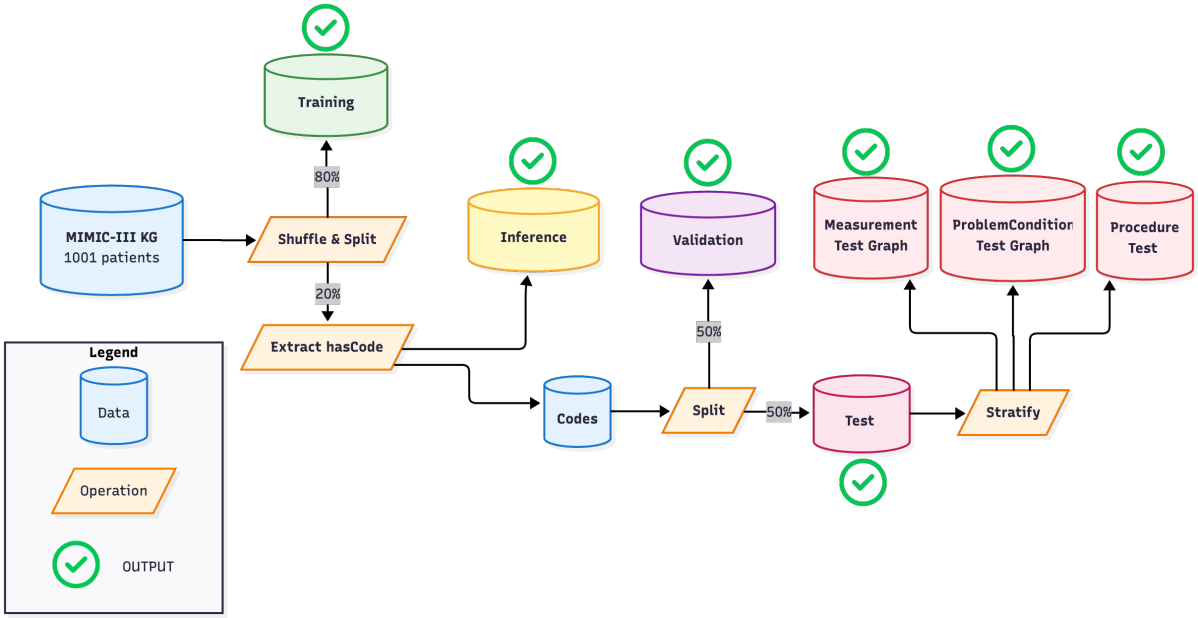

Figure 2: A diagram highlighting the high-level preprocessing steps before model training. Splitting is done by patient, e.g. one patient's data is never split between training and inference sets. Instead, the data is split by the whole patients.

3.2. Training

We use `InductiveNodePieceGNN` from PyKEEN[7] library, a graph neural network (GNN) architecture specifically designed to handle entities not present during training. Unlike traditional knowledge graph embedding models that learn a fixed vector representation for each entity, the model encodes local graph structure using the GNN encoder and scores candidate triples with DistMult. Training uses self-adversarial negative sampling (16 negatives per positive) and NSSALoss [19] (15 margin), optimized with `Adam` [24] (0.0005 learning rate). Early stopping monitors validation performance with a patience of five epochs to prevent overfitting.

3.3. Evaluation

Evaluation is performed exclusively on unseen patients, testing the model's inductive capability. For each missing code, the model ranks all candidate medical codes using filtered ranking, excluding trivially correct alternatives. Performance is reported using Hits@k ($k \in \{1, 5, 10\}$) and rank statistics under optimistic, realistic, and pessimistic tie-breaking assumptions. Results are further stratified by entity type (`Measurement`, `ProblemCondition`, `Procedure`) to identify domain-specific strengths and limitations. This evaluation setup reflects the intended deployment scenario, where a short ranked list of candidate codes is presented for human validation.

Ranking Variants Each test case produces a ranking of all candidate codes, and the evaluation computes multiple metrics to capture different aspects of ranking quality. Importantly, all metrics are calculated in three variants to account for very possible score ties:

- **Optimistic Ranking:** When multiple candidates receive identical scores, the correct answer is assumed to be ranked at the best (lowest) position among the tied group.
- **Realistic Ranking:** The correct answer is assigned the average rank among tied candidates.
- **Pessimistic Ranking:** The correct answer is assigned the worst (highest) position among tied candidates.

[7] https://pykeen.readthedocs.io/en/stable/

4. Results & Discussion

We present the performance evaluation of the inductive data imputation model for predicting medical codes. The model was tested on 18,953 clinical entity instances across three data types: laboratory tests (`Measurement`: 16,790 cases), diagnoses/observations (`ProblemCondition`: 1,089 cases), and medical operations (`Procedure`: 975 cases). Our evaluation examines the model's ability to suggest correct medical codes within the top-k recommendations. We ran the splitting + training pipeline five independent times and took the average of the results per setup. Therefore, all the analysis and plots given in this section could be considered an estimate of the general expected performance.

Ranking Variant Analysis We consider three ranking variants to account for tied prediction scores: Optimistic (best-case), realistic (average-case with random tie-breaking), and pessimistic (worst-case). Figure 3 illustrates the performance across these variants for the deployment scenario where users select the top-1 recommended code (Hits@1).

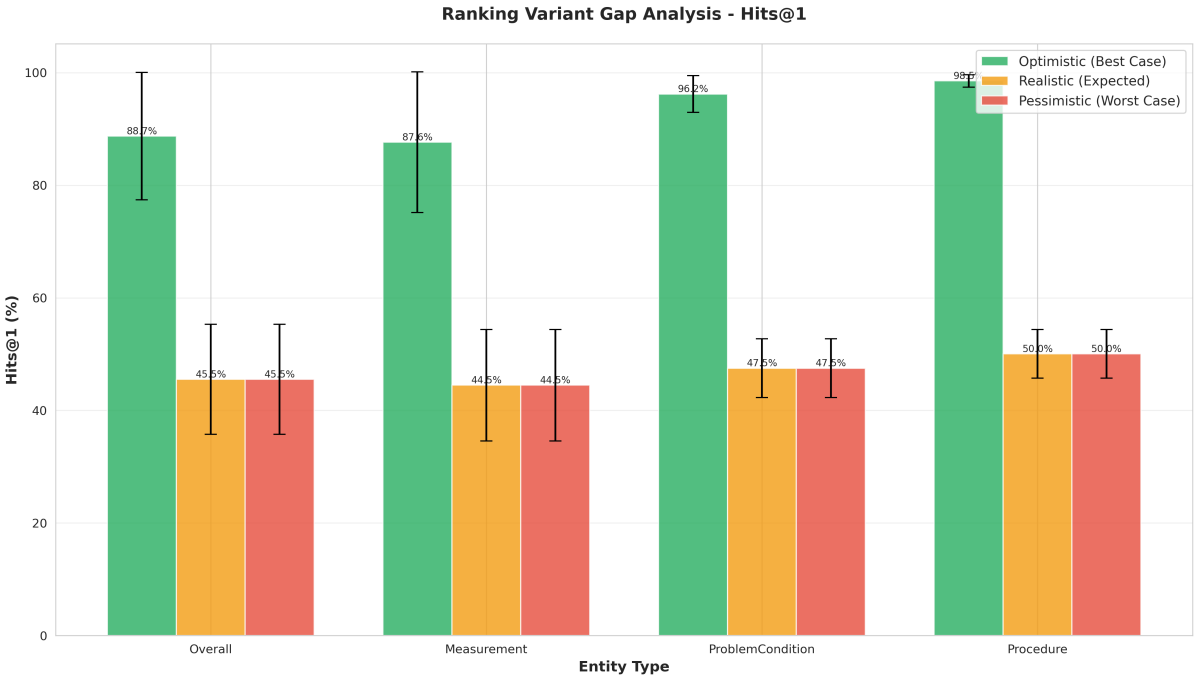

Figure 3: Hits@1 performance across ranking variants for each entity type. The gap between variants indicates the degree of tied scores in model predictions. Black lines indicate the standard deviation between 5 independent runs that were averaged.

The overall system achieves 88.7% of Hit@1 in finding the correct code at the top suggestion under optimistic ranking. Performance (Hit@1) drops substantially to 45.5% under realistic and pessimistic assumptions. This significant difference indicates that the model often assigns similar confidence scores to multiple alternatives rather than clearly separating the best one. Moreover, we see similar results when we investigate the performance for different entity types.

For `Measurement` entities, the model shows a comparable performance to the overall average for all scenarios (87.6% best case, 44.5% typical case, and 44.5% worst case). On the other hand, `ProblemCondition` and `Procedure` entities generally achieve over 95% Hits@1 under optimistic ranking.

From a practical standpoint, we focus on the optimistic ranking variant as the primary performance measure. This choice reflects how the intended use of the AIDAVA prototype in practice, where success is defined by the appearance of the correct code within the top-5 suggestions shown to the user. From a practical standpoint, if a user, e.g. patient or clinician or curator, can select the correct code from the list of 5 options, the system has successfully accomplished its goal. The differences between ranking

variants tell us about how confident the model is in distinguishing between options, but do not change whether the correct answer is provided to users. This confirms the practical viability of the approach for clinical curation scenarios where a small shortlist of candidate codes is presented to a human validator.

Performance by Entity Type Using the optimistic ranking as our main focus, Figure 4 shows how often the model has the correct answer included within the top-1, top-5, or top-10 suggestions.

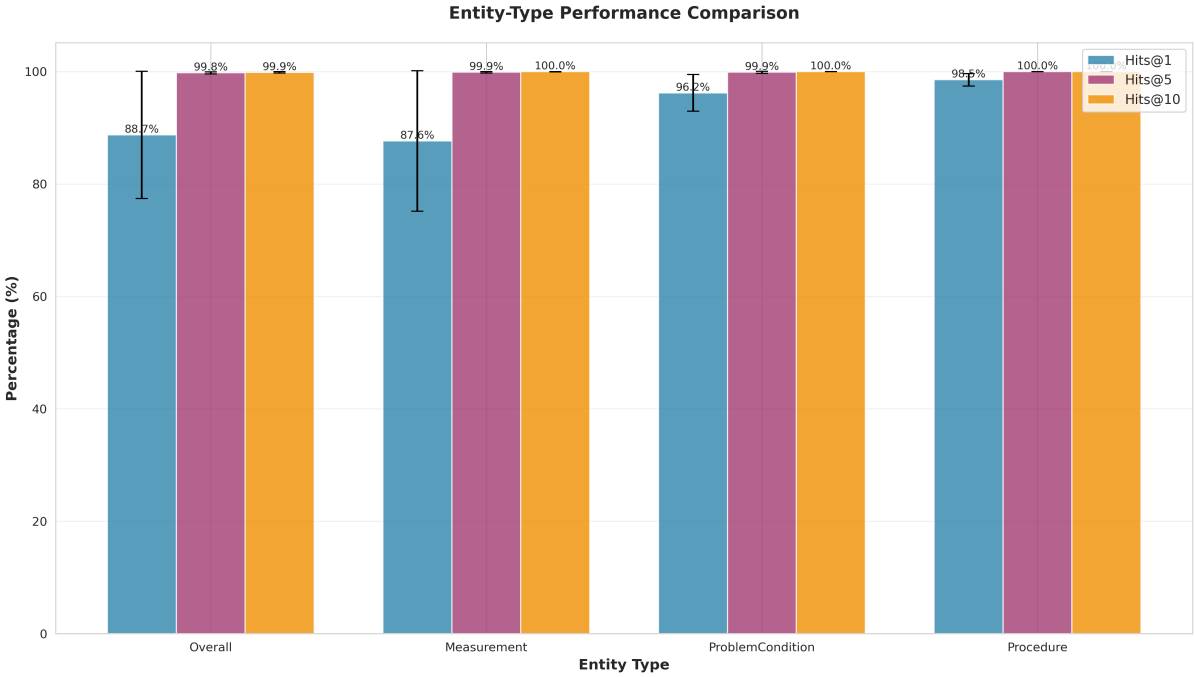

Figure 4: Hits@1, Hits@5, and Hits@10 performance across entity types using optimistic ranking. Performance varies substantially by clinical domain.

Across all entity types, the results show near-perfect Hits@5 and Hits@10 scores. Overall, the correct medical codes appear as the top-ranked suggestions in 88.7% of cases, and within the top-5 suggestions in 99.8% of cases. In practical terms, if the correct code does not appear as the first suggestion, it will almost always be found in positions 2 through 5. This supports the decision to show users five suggestions for practical deployment scenarios.

For `ProblemCondition` and `Procedure` entities, the correct diagnostic code appears first more than 95%, and appears within the top-5 suggestions nearly all of the time. Although these results appear to be optimistic, they also point to a larger problem regarding the data used to train and test this data imputation implementation.

Comparison versus Random Baseline & Data Bias To understand the consistency and reliability of the model's predictions, Figure 5 displays the range of performance across the five independent training runs we average. This plot shows three key values: the minimum performance observed in any run (bottom marker), the average performance across all runs (center circle), and the maximum performance achieved (top marker). The vertical lines connecting these points represent the full span of variation, helping us assess whether the model's performance is stable or highly dependent on random factors in training.

The overall performance accuracy for top suggestions shows a relatively wide range from 72.2% to 99.3%. This 27-percentage-point spread indicates moderate variability across training runs, suggesting that random initialization and data splitting have some impact on model performance. When examining individual entity types, we observe notably different patterns of consistency.

`Measurement` entities perform in a range spanning from 69.8% to 99.5%. This is a 30% variation that suggests this entity type's performance can vary substantially depending on the specific training

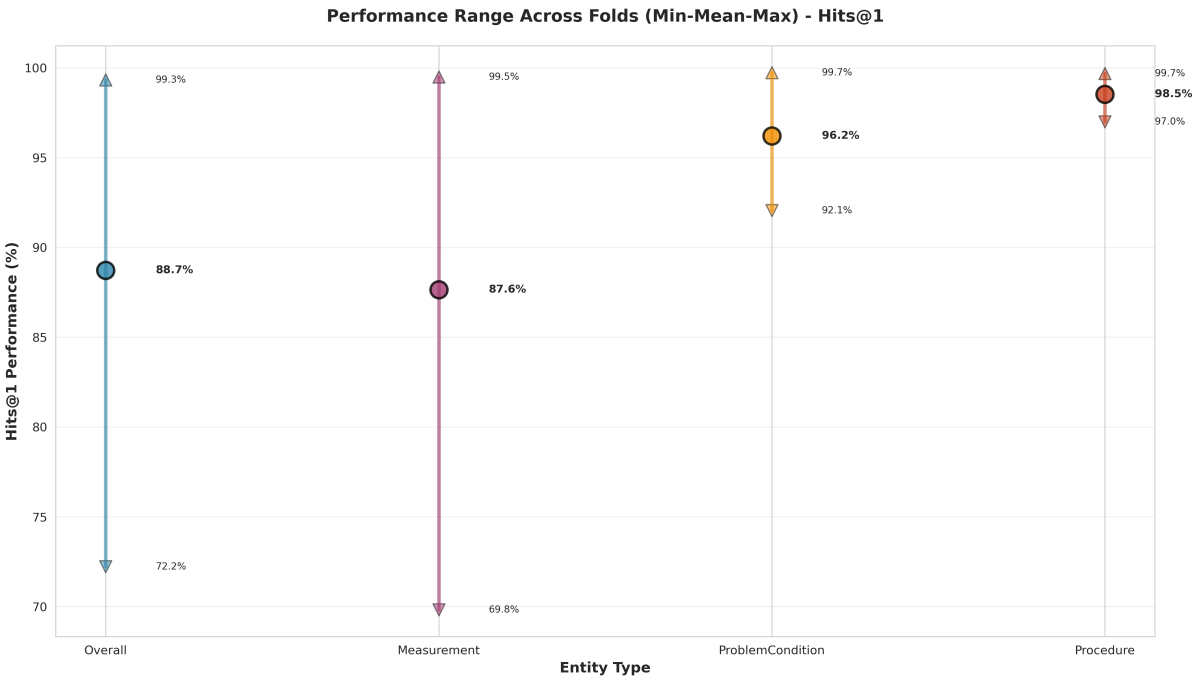

Figure 5: Performance range (minimum, mean, maximum) for Hits@1 across five cross-validation folds. The center circle shows the average performance, while the vertical lines indicate the best and worst performance observed across different training runs.

conditions. This finding also supports the notion that determining correct LOINC codes usually being a challenging task due to sites opting to use in-house coding for their laboratory tests [25]. In contrast, `ProblemCondition` entities has a narrower range (92.1% to 99.7%), indicating that the model has learned robust patterns for predicting diagnosis codes regardless of how the data is split. `Procedure` entities achieve the tightest range from 97.0% to 99.7%, demonstrating both excellent accuracy and high consistency across different training runs.

These performance ranges reveal that while the model generally performs well, `Measurement` predictions are more sensitive to training conditions than the other entity types. The narrow ranges for `ProblemCondition` and `Procedure` suggest these entity types have clearer, more learnable patterns in the data, whereas `Measurement` entities may benefit from additional training data or feature engineering to achieve more consistent predictions. This is directly related to how the medical codes are distributed within the source data, explored further below.

Figure 6 shows significant differences in how often specific codes appear in the training data. `Measurement` codes are frequent, with the median laboratory test appearing 31.5 times. In contrast, `ProblemCondition` codes are rare, appearing typically only 2 times. `Procedure` codes are the rarest, with a median frequency of just 1. This distribution explains the low standard deviation observed in Figure 5. `ProblemCondition` and `Procedure` entities show consistent performance across different training runs. Their limited variety likely creates simple patterns that the model learns reliably. However, `Measurement` performance varies widely (29.7% range). This suggests that the high frequency and diversity of lab tests force the model to learn complex boundaries. These boundaries shift significantly depending on how the training data is split.

We also observe a large gap (approximately 45%) between optimistic and realistic/pessimistic predictions in Figure 3. This indicates that the model identifies the correct code as a top candidate but fails to rank it first. It struggles to distinguish the specific correct answer from other plausible options. The reason for this ranking failure differs by entity type. For `Measurement` codes, many lab tests appear in similar clinical contexts, which makes them hard to tell apart. For `Procedure` codes, the data is too sparse. The model sees most procedures only once, so it cannot learn unique features to rank them confidently. Similarly, `ProblemCondition` codes often share overlapping symptoms. This ambiguity

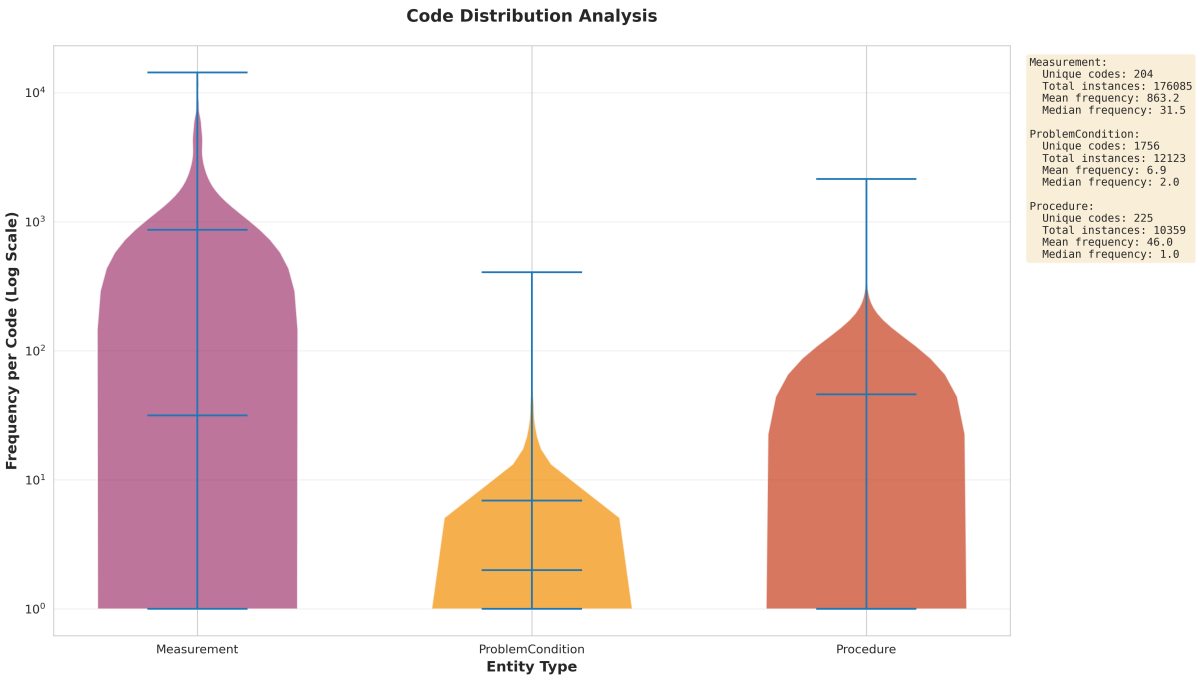

Figure 6: Distribution of code frequencies across entity types. The violin plot shows how often individual codes appear in the dataset, revealing differences in code reuse patterns.

prevents the model from assigning the highest confidence to the correct diagnosis. To fix this, the model requires more specific context to differentiate between these similar medical events.

Enriching with the ICD-9 Taxonomy To explore robust strategies, we enriched the 1001-patient MIMIC-III Knowledge Graph with the full ICD-9 taxonomy/ontology. Figure 7 compares the baseline model with the ICD-9 taxonomy-enriched model. This comparison reveals whether additional structural knowledge can help the model better distinguish between similar medical codes and improve its confidence calibration.

The results show a mixed but informative picture. The ICD-9 enrichment actually decreases optimistic performance from 88.7% to 76.6%, while improving realistic/pessimistic scores from 45.5% to 66.1%, narrowing the gap between optimistic and realistic narrows from 43% to just 10%. This indicates substantially improved confidence calibration at the cost of reduced top-rank accuracy.

The benefits are most pronounced for `Measurement` and `Procedure` entities. For `Measurement` type, realistic/pessimistic scores improve dramatically from 44.5% to 66.5%, leading to a 22% improvement while the optimistic score drops slightly from 87.6% to 77.8%. A similar effect is observed for `Procedure` entities, where the ranking shrinks from 48.5% to 10.9%. For both cases, the added complexity slightly reduces the breadth of its candidate sets, however, the model seems to learn patient characteristics better when it simply has more context surrounding its target code.

In contrast, `ProblemCondition` performance degrades under this enrichment. The optimistic score drops from 96.2% to 51.6%, while realistic scores remain stable (approximately 47-51%). This suggests that the large ICD-9 hierarchy (22,533 codes) creates too many competing pathways; essentially adding too much complexity to the ICD-9 section of the prediction space. The model can still rank specific codes well, but it cannot maintain a broad list of likely candidates. This difficulty contrasts with `Measurement` and `Procedure` entities, which only use the ICD-9 codes as context to learn patterns rather than predicting them directly.

Overall, ontology enrichment presents a clear trade-off. It improves ranking confidence but limits the breadth of the candidate sets. This result is favorable for `Measurement` and `Procedure` entities. However, for `ProblemCondition`, the added complexity is problematic. Through this study, we clearly observed that adding semantic information is not inherently beneficial and can backfire if not carefully

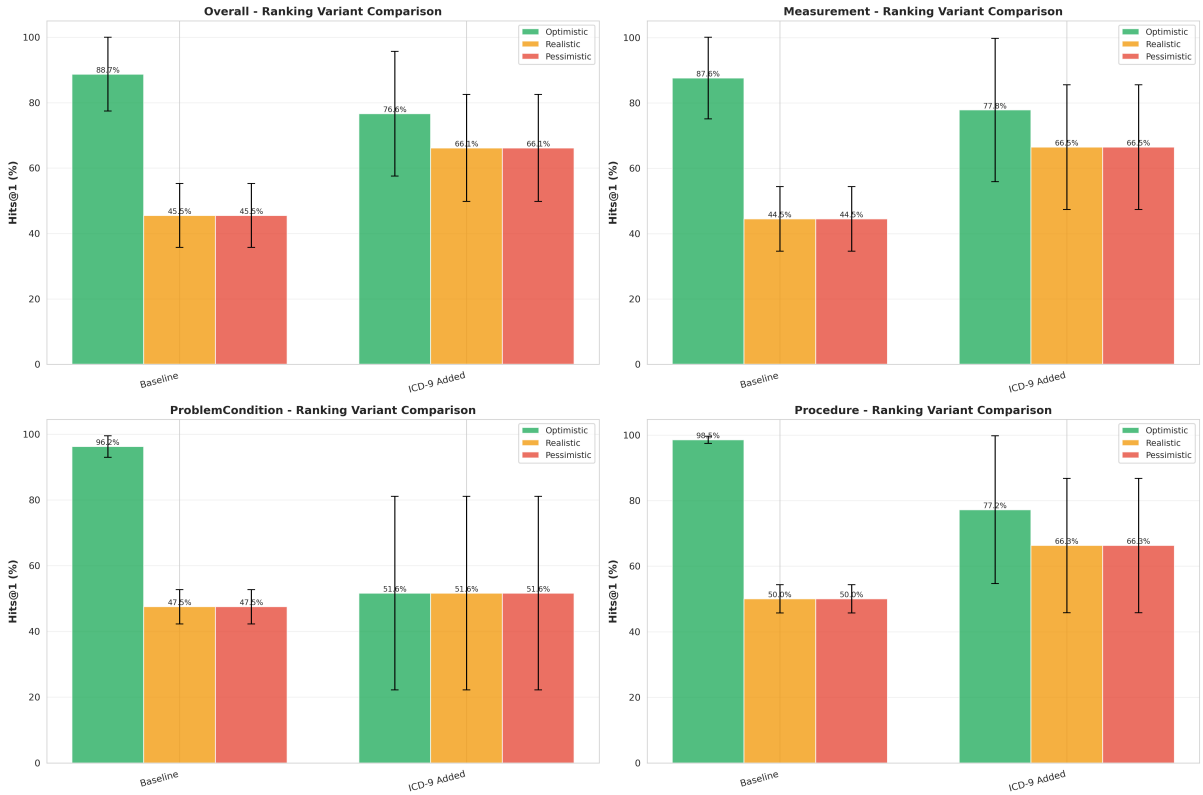

Figure 7: Hits@1 comparison between the baseline implementation and when ICD-9 taxonomy is added on top of the MIMIC-III Knowledge Graph. The results are given for all entity types, black bars indicate the standard deviation between 5 averaged runs.

curated. Main indication of this phenomenon is the performance dropping when the semantic input is too complex or closely related to the output variables, e.g. `ProblemCondition` having to predict from a more complex space after enriching with the ICD-9 taxonomy. Therefore, we can state that maximizing semantic volume will not always yield better deep learning predictions. Overall, enriching the model input with ICD-9 taxonomy suggests that deep learning approaches could benefit from even richer semantic data, including broader ontologies and synonym mappings. Provided the search space is controlled to limit noise, these resources could offer significant improvements on ranking tasks.

Limitations As with any ML system deployed in clinical settings, several risks and limitations must be acknowledged. Firstly, misaligned confidence scores are a major concern. The confidence scores show how certain the model is based on its training, but these scores might not perfectly match actual accuracy. For example, the `ProblemCondition` results show that the model can identify correct codes but cannot rank them confidently. As the model improves through more training and adjustment, its confidence scores will become more reliable. For now, predictions with lower confidence need extra validation.

A domain-specific limitation can be clearly observed in Baseline model's ranking variants' comparison (Figures 3 and 7). Although optimistic performance looks promising for Hits@1, the model cannot reliably assign confidence scores, hence the gap. This is mitigated by the enriching the MIMIC-III Knowledge Graph with the entire ICD-9 terminology. However, this only manages to reduce the gap for `Measurement` and `Procedure` entities. `ProblemCondition` performance is reduced by this modification due to its prediction range becoming more complex. This highlights the shortcomings of the domain-specific data we use; specifically MIMIC-III being too well-curated for an EHR compared to the real-world data, and our conversion to a knowledge graph being too structurally-sound. Therefore, a live implementation of this model should clearly warn users about the imputation predictions being

biased on a limited set of instances. On the other hand, better enhancements should be explored to remedy the ranking issue, such as gathering more diverse `ProblemCondition` training data, adding more context about the entity types, or fine-tuning the model to predict for certain rare codes.

Performance variability across entity types creates another design challenge. Users might expect all entity types' predictions to be of the same quality, then feel confused when, e.g. `ProblemCondition`, suggestions are less reliable or recognizable for their context. Ideally, a data imputation model should be fully-automated, ranking the correct medical code at the top each time. However, due to risks unique to the healthcare domain, human involvement is required almost always; even when a all outputs seem perfect. Otherwise, even a single mis-coding can cascade into problems that could affect patient well-being. Therefore, clear communication about how well each entity type works alongside confidence thresholds during human review can manage user expectations and maintain trust in the system.

Finally, building separate models for each entity type can reduce the variability. This is because the current model implements a holistic, fully-inductive process where all medical codes are hidden during inference. This limits the learning achieved during the validation loop as none of the predicted links are able to draw from other entities' codes, leading to possibly miscalculating the loss value after each iteration.

5. Conclusion & Future Improvements

In this study, we presented a link prediction implementation designed to impute data for missing medical codes. A central methodological contribution of this work is the shift from transductive to inductive learning: rather than relying on fixed vector representations that require retraining for every new entity, our model dynamically constructs representations from constituent tokens and graph relationships. This approach is critical for the healthcare domain, where registries are highly dynamic, characterized by the continuous admission of new patients and the evolution of medical terminologies.

Experimental results demonstrate that the model achieves promising results across all entity types (e.g. `Measurement`, `ProblemCondition`, and `Procedure`), consistently ranking correct codes highly within the candidate space. However, we observed a notable discrepancy between ranking variants, indicating that the model's confidence scoring required calibration. To address this, we integrated the MIMIC-III Knowledge Graph with the ICD-9 ontology to enrich the available semantic context. This intervention yielded promising improvements for `Measurement` and `Procedure` entities, reducing the divergence between ranking variants by approximately 40%. Conversely, for `ProblemCondition` entities, the expansion of the search space with additional ICD-9 codes introduced significant complexity. The model, lacking patient-specific context for these newly introduced ontological nodes, struggled to differentiate between viable alternatives, resulting in a performance drop in the optimistic ranking variant from 95% to 53%.

Future efforts to strengthen this approach can focus on five key areas. First, to help the model recognize more than just the specific codes it saw during training, it would be beneficial to integrate a wider range of data sources. Including lists of synonymous terms and structured medical hierarchies would allow the system to better understand how different concepts connect. Second, the difficulty in predicting `ProblemCondition` entities after injecting the ICD-9 ontology suggests that the training process may need refinement. Improved strategies for selecting training examples could help the model learn patterns even when data for certain conditions is rare or repetitive. Third, the confidence score the model outputs should match its actual accuracy. Post-training calibration, such as temperature scaling or isotonic regression, can align raw confidence scores to establish reliable thresholds for human review in live implementation of this model. Fourth, a human-in-the-loop mechanism could be developed to let the model learn from real-world usage. By observing which suggestions are accepted or rejected by coders, the system could continuously improve and adapt to new medical codes in a more active manner. Finally, to mitigate possible explainability concerns with this deep learning implementation, it is important to derive information from the model's output for why a recommendation was made and

also monitor performance across different patient groups to ensure fairness. These advancements aim to evolve the current framework, which already achieves an overall 88.7% Hits@1; into a comprehensive, equitable, and transparent solution for imputing missing medical codes.

Acknowledgments

This work is supported by the Horizon Europe Framework Program under Grant Agreement No. 101057062 (AIDAVA).

Declaration of AI Use Generative AI tools were used to draft certain sections of the initial manuscript and assist with the Python code for visualizations.

References

[1] N. G. Weiskopf, C. Weng, Methods and dimensions of electronic health record data quality assessment: enabling reuse for clinical research, Journal of the American Medical Informatics Association 20 (2013) 144–151. URL: https://doi.org/10.1136/amiajnl-2011-000681. doi:10.1136/amiajnl-2011-000681. arXiv:https://academic.oup.com/jamia/article-pdf/20/1/144/9517051/20-1-144.pdf.

[2] B. J. Wells, K. M. Chagin, A. S. Nowacki, M. W. Kattan, Strategies for handling missing data in electronic health record derived data, EGEMS (Wash., DC) 1 (2013) 1035.

[3] Z. Che, S. Purushotham, K. Cho, D. Sontag, Y. Liu, Recurrent neural networks for multivariate time series with missing values, Scientific Reports 8 (2018) 6085. URL: https://doi.org/10.1038/s41598-018-24271-9. doi:10.1038/s41598-018-24271-9.

[4] Q. Wang, Z. Mao, B. Wang, L. Guo, Knowledge graph embedding: A survey of approaches and applications, IEEE Transactions on Knowledge and Data Engineering 29 (2017) 2724–2743. doi:10.1109/TKDE.2017.2754499.

[5] K. Teru, E. Denis, W. Hamilton, Inductive relation prediction by subgraph reasoning, in: H. D. III, A. Singh (Eds.), Proceedings of the 37th International Conference on Machine Learning, volume 119 of *Proceedings of Machine Learning Research*, PMLR, 2020, pp. 9448–9457. URL: https://proceedings.mlr.press/v119/teru20a.html.

[6] M. Galkin, M. Denis, J. Wu, W. L. Hamilton, Nodepiece: Compositional and parameter-efficient representations of large knowledge graphs, 2022. URL: https://arxiv.org/abs/2106.12144. arXiv:2106.12144.

[7] A. E. W. Johnson, T. J. Pollard, L. Shen, L.-W. H. Lehman, M. Feng, M. Ghassemi, B. Moody, P. Szolovits, L. Anthony Celi, R. G. Mark, Mimic-iii, a freely accessible critical care database, Scientific Data 3 (2016) 160035.

[8] P. B. Jensen, L. J. Jensen, S. Brunak, Mining electronic health records: towards better research applications and clinical care, Nature Reviews Genetics 13 (2012) 395–405. URL: https://doi.org/10.1038/nrg3208. doi:10.1038/nrg3208.

[9] D. Agniel, I. S. Kohane, G. M. Weber, Biases in electronic health record data due to processes within the healthcare system: retrospective observational study, BMJ 361 (2018). URL: https://www.bmj.com/content/361/bmj.k1479. doi:10.1136/bmj.k1479. arXiv:https://www.bmj.com/content/361/bmj.k1479.full.pdf.

[10] K. Bhaskaran, L. Smeeth, What is the difference between missing completely at random and missing at random?, Int. J. Epidemiol. 43 (2014) 1336–1339.

[11] J. Digitale, D. Franzon, M. J. Pletcher, C. E. McCulloch, E. D. Gennatas, Methods for addressing missingness in electronic health record data for clinical prediction models: Comparative evaluation, JMIR Med Inform 13 (2025) e79307. URL: https://medinform.jmir.org/2025/1/e79307. doi:10.2196/79307.

[12] K. Getz, R. A. Hubbard, K. A. Linn, Performance of multiple imputation using modern machine learning methods in electronic health records data, Epidemiology 34 (2023) 206–215.

[13] N. Natarajan, I. S. Dhillon, Inductive matrix completion for predicting gene–disease associations, Bioinformatics 30 (2014) i60–i68. URL: https://doi.org/10.1093/bioinformatics/btu269. doi:10.1093/bioinformatics/btu269. arXiv:https://academic.oup.com/bioinformatics/article-pdf/30/12/i60/48927398/bioinformatics_30_12_i60.pdf.

[14] P. J. García-Laencina, J.-L. Sancho-Gómez, A. R. Figueiras-Vidal, Pattern classification with missing data: a review, Neural Computing and Applications 19 (2010) 263–282.

[15] J. Yoon, J. Jordon, M. van der Schaar, Gain: Missing data imputation using generative adversarial nets, 2018. URL: https://arxiv.org/abs/1806.02920. arXiv:1806.02920.

[16] D. Saxena, J. Cao, Generative adversarial networks (gans): Challenges, solutions, and future directions, ACM Comput. Surv. 54 (2021). URL: https://doi.org/10.1145/3446374. doi:10.1145/3446374.

[17] R. Miotto, F. Wang, S. Wang, X. Jiang, J. T. Dudley, Deep learning for healthcare: review, opportunities and challenges, Briefings in Bioinformatics 19 (2017) 1236–1246. URL: https://doi.org/10.1093/bib/bbx044. doi:10.1093/bib/bbx044. arXiv:https://academic.oup.com/bib/article-pdf/19/6/1236/27119191/bbx044.pdf.

[18] A. Bordes, N. Usunier, A. Garcia-Durán, J. Weston, O. Yakhnenko, Translating embeddings for modeling multi-relational data, in: Proceedings of the 27th International Conference on Neural Information Processing Systems - Volume 2, NIPS'13, Curran Associates Inc., Red Hook, NY, USA, 2013, p. 2787–2795.

[19] Z. Sun, Z.-H. Deng, J.-Y. Nie, J. Tang, Rotate: Knowledge graph embedding by relational rotation in complex space, 2019. URL: https://arxiv.org/abs/1902.10197. arXiv:1902.10197.

[20] B. Yang, W.-t. Yih, X. He, J. Gao, L. Deng, Embedding entities and relations for learning and inference in knowledge bases, 2015. URL: https://arxiv.org/abs/1412.6575. arXiv:1412.6575.

[21] T. Trouillon, J. Welbl, S. Riedel, É. Gaussier, G. Bouchard, Complex embeddings for simple link prediction, 2016. URL: https://arxiv.org/abs/1606.06357. arXiv:1606.06357.

[22] S. Zhang, Y. Tay, L. Yao, Q. Liu, Quaternion knowledge graph embeddings, 2019. URL: https://arxiv.org/abs/1904.10281. arXiv:1904.10281.

[23] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg, G. Appleton, M. Axton, A. Baak, N. Blomberg, J.-W. Boiten, L. B. da Silva Santos, P. E. Bourne, J. Bouwman, A. J. Brookes, T. Clark, M. Crosas, I. Dillo, O. Dumon, S. Edmunds, C. T. Evelo, R. Finkers, A. Gonzalez-Beltran, A. J. Gray, P. Groth, C. Goble, J. S. Grethe, J. Heringa, P. A. 't Hoen, R. Hooft, T. Kuhn, R. Kok, J. Kok, S. J. Lusher, M. E. Martone, A. Mons, A. L. Packer, B. Persson, P. Rocca-Serra, M. Roos, R. van Schaik, S.-A. Sansone, E. Schultes, T. Sengstag, T. Slater, G. Strawn, M. A. Swertz, M. Thompson, J. van der Lei, E. van Mulligen, J. Velterop, A. Waagmeester, P. Wittenburg, K. Wolstencroft, J. Zhao, B. Mons, The fair guiding principles for scientific data management and stewardship, Scientific Data 3 (2016) 160018. URL: https://doi.org/10.1038/sdata.2016.18. doi:10.1038/sdata.2016.18.

[24] D. P. Kingma, J. Ba, Adam: A method for stochastic optimization, 2017. URL: https://arxiv.org/abs/1412.6980. arXiv:1412.6980.

[25] M. Stram, J. Seheult, J. H. Sinard, W. S. Campbell, A. B. Carter, M. E. de Baca, A. M. Quinn, H. S. Luu, A survey of loinc code selection practices among participants of the college of american pathologists coagulation (cgl) and cardiac markers (crt) proficiency testing programs, Archives of Pathology & Laboratory Medicine 144 (2020) 586–596. URL: https://doi.org/10.5858/arpa.2019-0276-OA. doi:10.5858/arpa.2019-0276-OA.